## Weeks 1–2: Analysis, Research, and State of the Art:

**Objective of these two weeks:**

To understand the domain of AI-assisted software testing, analyze existing tools, define the functional and technical requirements of the project, and make the first architectural decisions before writing a single line of code.

***Week 1(starting 02/02/2026 until 06/02/2026) Domain analysis and study of existing solutions:***

During the first week of my internship, my main objective was to understand the domain of AI-assisted software testing and analyze the existing platforms before starting the technical development of my project. I conducted an in-depth study of several test management and AI testing tools such as TestRail, qTest, Zephyr, and TestMu AI (KaneAI). I particularly analyzed KaneAI by examining screenshots and official documentation in order to understand how its system manages test steps and executes tests in a live browser. Through this analysis, I identified the functional scope of my project, which combines test management, Ai test scenarios generation and AI-based execution. I also researched the different types of tests involved in such platforms, including end-to-end (E2E), functional, and acceptance tests. This work allowed me to draft the first version of the functional specification document for my platform. During this process, I learned about the complete lifecycle of a test, from the initial specification to the generation of XML artifacts used in CI/CD pipelines. I also understood the important distinction between test generation using natural language processing and autonomous execution using browser automation and visual tools. One key design principle that emerged is that the manual testing mode must remain fully functional independently of AI, with AI acting as an additional intelligent layer. One difficulty I encountered was the large number of AI testing tools available, which made the analysis phase more complex than expected. To overcome this challenge, I created a structured comparison grid to objectively evaluate the different platforms. Finally, discovering that tools such as KaneAI can execute tests in a real browser in real time gave me a clearer vision of the platform I aim to build, where the integration between test management and autonomous AI execution represents the core value of the project.

***Week 2 (from 09/02/2026 to 13/02/2026) Architectural Decisions and Technology Choices :***

During the second week of my internship, I focused on making the main architectural and technological decisions for the project. I first selected and justified the technical stack that will be used to develop the platform, which includes Django for the backend, React for the frontend, LangGraph/LangChain for agent orchestration, and PostgreSQL as the database. I also defined the AI strategy by comparing two possible approaches: using a hybrid RAG + LLM API architecture or fine-tuning a model. After analyzing the needs of the project, I decided that RAG combined with LLM APIs is sufficient for the MVP, since modern language models already understand most QA concepts. I also chose the models that will be used during development and production, such as Gemini Flash and Llama 3.3 for free experimentation during development, and Claude Sonnet for production. Another important decision was to design the system as a Django monolith with LangGraph integrated as an internal application instead of building a separate microservice. I also decided to use LiteLLM (may change based on needs) as an abstraction layer to easily switch between LLM providers without modifying the core code. During this phase, I learned more about software architecture decisions, particularly the trade-offs between microservices and monolithic architectures. I also explored how asynchronous task

processing works using Celery, which allows AI agents to run in the background without blocking the Django application. In addition, I discovered that integrating pgvector directly into PostgreSQL avoids the need for a separate vector database service. One difficulty I faced was determining whether LangGraph should be deployed as a separate service or directly integrated into Django. To solve this problem, I conducted a comparative analysis and concluded that a Django monolith with an internal agents module is sufficient for this project. At first, the decision not to fine-tune a model seemed risky, but after carefully analyzing the features required for the MVP, I realized that the combination of RAG and LLM APIs already covers all the necessary functionalities. Fine-tuning can be introduced later once users generate enough annotated corrections and feedback to justify training a specialized model until then the usage of context windows and Memory combined with the vector database could serve as a good way to update the model output.